RESUMEN-CUESTIONARIO-1--BATERIA.pdf

jggomezuca

Programación Orientada a Objetos

2º Grado en Ingeniería Informática

Escuela Superior de Ingeniería
Universidad de Cádiz

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Cheetos
MÁNCHATE LOS DEDOS CON CHEETOS
Y ESCAQUÉATE
DE TOMAR APUNTES HOY

EVOLUCIÓN HISTÓRICA Y PRINCIPIOS FUNDAMENTALES

Introducción al concepto de programación orientada a objetos

- Puede verse como una **evolución de la programación estructurada**

- Plantea un **nuevo paradigma de programación**

- Enfoque de **programación con más relevancia** actualmente

Cronología (POO)

- **Años 60** → Desarrollo del lenguaje de programación Simula 67, **aparición del concepto de objeto**. - **Años 70** → Desarrollo del lenguaje de programación SmallTalk (Xerox PARC) - **Años 80** → Desarrollo del lenguaje de programación C++ (AT&T)

- **Años 90** → Desarrollo del lenguaje de programación Java (Sun Microsystems)

Propiedades básicas de los lenguajes orientados a objetos

- DEBEN estar **basados en objetos y clases**

- DEBEN usar **técnicas basadas en la herencia y el polimorfismo**

Los únicos lenguajes de programación que cumplen estos requisitos son C + + y Java.

JavaScript → Basado en objetos, pero no orientado a objetos
CLU → Basado en clases, pero no orientado a objetos
SmallTalk y Eiffel → Otras propiedades (más puros)

Cronología de la historia de C + +

- 1985 → Creación del C++ (Bjarne Stroustrup, Laboratorios Bell AT&T)

- 1989 → Comité X3J16 de ANSI para su estandarización

- 1997 → Publicación del estándar ANSI de C + +

- 1998 → Publicación del estándar ISO de C++ (C + + 98)

- 2003 → Technical corrigendum del ISO C + +

- 2011 → Nuevo estándar ISO C + + (C + + 11)

Características del C + +

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.
WUOLAH

- Supone un **compromiso entre** la **programación** a **bajo y alto nivel**

- Es un **lenguaje multiparadigma** (no es puramente orientado a objetos)

¿Qué son los paradigmas de programación?

En resumidas cuentas, es **un estilo o manera de programación**.

Existen diferentes formas de diseñar un lenguaje de programación y varios modos de trabajar para obtener los resultados que necesitan los programadores.

Técnicamente se trata de un conjunto de métodos sistemáticos aplicables en todos los niveles del diseño de programas para resolver problemas computacionales.

Paradigmas del lenguaje de programación C + +

- Programación **estructurada**

- Programación **orientada a objetos**

- Programación **genérica**

Características de la orientación a objetos

- Sistemas → Organizados en torno a objetos (no a datos y procesos)

- Nuevo **concepto de los sistemas en los lenguajes orientados a objetos** → Conjunto de objetos

Componentes y funcionamiento de los objetos

- Los objetos **tienen características y comportamientos**

- Los objetos **se comunican entre sí mediante mensajes**

Metodología para plantear un problema usando un lenguaje orientado a objetos

- Identificar los objetos a crear

- Agrupar los objetos con características y comportamientos comunes (para agruparlos en

clases) - Identificar las clases a crear que recojan dichas características y comportamientos. -

Identificar las relaciones que existen entre las clases (y cómo se comunican entre sí) - Identificar

los mensajes a pasar

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Participa en competiciones de programación. Encuentra el trabajo de tus sueños con NUWE

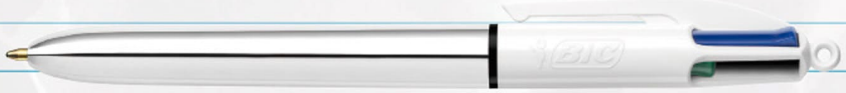
BIC®

4 Colours®

UN BOLÍGRAFO

PARA CADA

UNO DE TUS



ESTILOS.

TUS NOTAS O TU PLAN PARA EL FINDE

#

MÁS COLORES. MÁS DISEÑOS.

ELIGE UN BOLÍGRAFO QUE DIGA MÁS DE TI QUE TU ESCRITURA.

Principios fundamentales de la programación orientada a objetos

- **Abstracción** → Facilita la reutilización (Ficheros cabecera [.h] y otros [.cpp])

- **Encapsulado** → Aumenta la cohesión del código (Clases y estructuras)

- **Ocultación de la información** → Reduce el acoplamiento (Mediante atributos públicos y privados)

- **Generalización** → Disminuye la redundancia (Agrupación de objetos en una misma clase) -

Polimorfismo → Aumenta la capacidad de adaptación y extensibilidad

Ventajas del enfoque orientado a objetos

- **Simplicidad del modelo (facilidad de uso)**
    - Cercanía de sus conceptos a los del mundo real
    - Uso de los mismos elementos en todo el proceso de desarrollo

- **Reutilización** → Proceso de desarrollo más sencillo y rápido

- **Facilidad de mantenimiento** → Modificaciones, extensiones y adaptaciones más sencillas

- **Fiabilidad** → Corrección y robustez

Limitaciones del enfoque orientado a objetos

- Rechazo inicial por la dificultad de adaptación

- Análisis incorrectos

- Lenguajes incompletos

- Menos eficiencia

Factores calidad del software

- **Corrección** → Capacidad de los sistemas software de realizar con exactitud sus tareas (especificación)

- **Robustez** → Capacidad de los sistemas software de reaccionar apropiadamente ante condiciones excepcionales

- **Extensibilidad** → Capacidad de adaptar los sistemas a los cambios de especificación

- **Reutilización** → Capacidad de los componentes de los sistemas software de servir para la construcción de aplicaciones diferentes.

- **Compatibilidad** → Capacidad de combinar unos componentes con otros

No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.
Reservados todos los derechos.

WUOLAH

Participa en competiciones de programación. Encuentra el trabajo de tus sueños con NUWE

- **Eficiencia** → Capacidad de los sistemas software para exigir la menor cantidad posible de recursos hardware (tiempos de CPU, espacio en memoria)

- **Portabilidad / Transportabilidad** → Facilidad de transferir los sistemas software a diferentes entornos

- **Facilidad de uso** → Facilidad con la cual personas pueden aprender a usar el software y aplicarlo a la resolución de problemas

- **Funcionalidad** → Conjunto de posibilidades que proporciona un sistema software

- **Oportunidad** → Capacidad de un sistema software de ser aceptado en el mercado

- **Integridad** → Capacidad de los sistemas software de proteger sus componentes contra modificaciones o accesos no autorizados.

- **Economía** → Capacidad de un sistema para completarse con el presupuesto - **Facilidad de mantenimiento** → Facilidad de un sistema para poder depurarse y modificarse.

Aspectos claves del desarrollo orientado a objetos

- **Método y lenguaje** → Enfoque y notaciones que se deben emplear para analizar y producir software orientado a objetos.

- **Implementación y entorno** → Herramientas que permiten a los desarrolladores aplicar la tecnología orientada a objetos.

- **Bibliotecas** → Mecanismos que se deben emplear en el enfoque orientado a objetos a objetos para la utilización y producción de bibliotecas.

Definición: Una biblioteca es un conjunto de implementaciones funcionales, codificadas en un lenguaje de programación, que ofrece una interfaz bien definida para la funcionalidad que se requiere.

Métodos y lenguajes orientados a objetos

- **Clases** → Concepto central. Plantilla que define la forma de un objeto; en ella se agrupan datos y métodos que operarán sobre esos datos.

- **Módulos** → Pequeñas unidades de código independientes y reutilizables. Las clases deben ser los únicos módulos

- **Tipos** → Basados en una clase. Los tipos indican la naturaleza de los datos que se manejan en el programa. Hay tipos de datos básicos predefinidos (int, boolean, double, …) y otros definidos mediante las clases.

- **Base de la computación** → Paso de mensajes (Comunicación entre objetos)

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Participa en competiciones de programación. Encuentra el trabajo de tus sueños con NUWE

Cheetos
MÁNCHATE LOS DEDOS CON CHEETOS
Y ESCAQUÉATE
DE TOMAR APUNTES HOY

- **Ocultación de la información** → Abstracción

- **Asertos** → Monitorización y generación de documentación. Es una condición que se debe cumplir en un determinado punto del programa, de forma que el programa abortará en ese punto si no se cumple.

- **Excepciones** → Mecanismo de control de errores. Situaciones anómalas que pueden ocurrir durante la ejecución de las aplicaciones.

- **Genericidad** → Técnicas que permiten escribir algoritmos o definir contenedores de forma que puedan aplicarse a un amplio rango de tipos de datos.

- **Genericidad no restringida** → Clases parametrizadas aplicable a cualquier tipo de dato.

- **Herencia** → Permite que se puedan definir nuevas clases basadas de unas ya existentes a fin de reutilizar el código, generando así una jerarquía de clases dentro de una aplicación. Si una clase se deriva de otra, esta hereda sus atributos y métodos y puede añadir nuevos atributos, métodos o redefinir los heredados.

- **Genericidad restringida** → Mediante la herencia. Existen algoritmos que no se pueden aplicar a cualquier tipo de datos. Al restringir los tipos obtenemos el beneficio de poder aplicar métodos sobre los objetos del tipo parametrizado.

- **Polimorfismo** → Redefiniciones y conversiones. Propiedad por la que es posible enviar mensajes sintácticamente iguales a los objetos de tipos distintos. El único requisito que deben cumplir los objetos polimórficos es saber responder al mensaje que se les envía.

- **Ligadura dinámica** → Como consecuencia del polimorfismo, la forma del objeto determina la versión de la operación que se aplicará.

- **Comprobación estática de tipos y en tiempo de ejecución**

- **Gestión de memoria y recolección de basura** → La recolección de basura libera al programador de realizar gestión de memoria manual donde el programador especifica qué objetos desasignar y regresar al sistema de memoria y cuándo hacerlo.

- **Ciclo de vida** → Los métodos OO pueden aplicarse en todo el ciclo de vida

Implementación y entorno

- Actualización automática → El análisis de las dependencias debe efectuarse con herramientas

- Actualización rápida → El tiempo necesario para procesar un cambio debe estar en función del tamaño de lo modificado, y no del tamaño del sistema total.

- Persistencia → Debe disponerse de un mecanismo de almacenamiento de los objetos

- Documentación → Debe disponerse de herramientas automáticas para su generación

- Navegación → Debe disponerse de herramientas de navegación interactivas

No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.
Reservados todos los derechos.
WUOLAH

Bibliotecas

- Bibliotecas básicas → Estructuras de datos y algoritmos más frecuentes

- Bibliotecas genéricas → Interfaces gráficas de usuario y bases de datos

- Mecanismos de indexación de las bibliotecas → Para permitir una recuperación basada en propiedades.

Libro Fundamentos C++

- Finales década 60 → Primera propuesta de nueva forma de programación:
  Desarrollo de software de forma estructurada y modular.
  Posibilidades de abstracción y reutilización

- Primeros lenguajes orientados a objetos → SMALLTALK

- Primeras implementaciones SMALLTALK → Lenguaje interpretado (lento debido a la poca potencia de cálculo)

- Mediados década 80 → Nacimiento C++ (1985)

- Creador C++ → Bjarne Stroustrup, laboratorios Bell, AT&T

- Significado C++ → Operador incremento de C, aplicado a su propio nombre (C++ → "C incrementado")

- Antecesor C → Lenguaje B, derivado de BCPL (Basic Combined Programming Language) - Creador B

→ Dennis Ritchie

- Propósito B → Escribir S.O Unix en un lenguaje de más alto nivel que ensamblador

- Origen nombre UNIX → Derivación de UNICS (simplificación de un proyecto anterior de un S.O llamado MULTICS que pretendía controlar una ciudad entera y fue abandonado).
- Éxito C → Distribución gratuita de UNIX junto con un compilador de C y el código fuente. A partir de este éxito, surge la necesidad de adaptarlo a más propósitos de aquellos para los cuales fue concebido.

- Creación del estándar ANSI de C → Creado por el comité X3J11 de ANSI. Solucionó el problema de los dialectos de C (1989)

- Estándar ISO C → Equivalente al ANSI, pero ligeramente ampliado para facilitar su empleo en países de habla no inglesa.

- iso646.h → (1994) Anexo del estándar ISO con ampliaciones menores. Conocido como propuesta danesa.

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Participa en competiciones de programación. Encuentra el trabajo de tus sueños con NUWE

- Si alguien quiere que su programa C sea entendido por cualquiera que sepa C, o que se pueda transportar a máquinas distintas debe emplear C ANSI / ISO

- Problema C → Para resolver ciertos problemas de programación, este lenguaje se queda corto.

- Solución al problema de C → Desarrollo de lenguajes derivados; C concurrente, C objetivo, C paralelo, C + +...

Tipo de lenguaje C++ → Orientado a objetos, aunque no puro (no es puro porque soporta otros estilos de programación como el estructurado)

Otra nomenclatura del tipo de lenguaje C++ → Lenguaje híbrido o lenguaje orientado a objetos no puro

Ventajas del tipo de lenguaje C + + → Según su creador, el hecho de que no sea puro es más ventaja que inconveniente, ya que lo hace más versátil y adecuado para un mayor número de aplicaciones.

- Creación del estándar ANSI de C + + → Creado por el comité X3J16 de ANSI. Solucionó el problema de los dialectos de C + + (1989 - 1997)

- Si alguien quiere que su programa C + + sea entendido por cualquiera que sepa C + +, o que se pueda transportar a máquinas distintas debe emplear C + + ANSI / ISO.

- En sus orígenes C++ incorporó conceptos ya existentes de otros lenguajes.

- Concepto de clase → Inicialmente introducido en SIMULA67 (lenguaje orientado a la simulación de procesos)

- Primeras versiones de C++ → Extensiones de C (C con clases) con el objetivo de ser empleado en simulación.

Evolución C++ → C++ ha conseguido ser un lenguaje de programación de propósito general, ligeramente sesgado hacia la programación de sistemas.

Ventajas C++ → Avance respecto al lenguaje C (se considera como un C mejorado) Facilita la

abstracción de datos y operaciones a través del paradigma de la POO.

Permite también trabajar con los paradigmas de la programación estructurada y genérica, e incluso mezclar los tres (estructurada, genérica y POO).

Visión paradigma de la orientación a objetos → Más amplia e integrada del proceso de desarrollo software.

Software en un sistema orientado a objetos → Conjunto finito de objetos que contienen tanto datos como operaciones y que se comunican entre sí mediante mensajes.

Pasos para la modelación de un sistema empleando orientación a objetos:
- Identificar los objetos que intervienen en él.
- Agrupar en clases a todos aquellos objetos que tengan características y comportamientos comunes. - Identificar los datos y operaciones de cada una de las clases.

No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.
Reservados todos los derechos.

WUOLAH

Participa en competiciones de programación. Encuentra el trabajo de tus sueños con NUWE

- Identificar las relaciones que puedan existir entre las clases.

- Objeto → Elemento individual con su propia identidad
- Clase → Colección de objetos similares con características y comportamientos propios de la clase.

Relaciones entre clases:
- Asociación → Relación estructural entre objetos (Autor → Libro)
- Agregación → Caso particular de asociación. Relación entre un todo y sus partes (Libro → Capítulo)
- Generalización → Superclase de un conjunto de objetos con comportamientos y características comunes (Autor → Persona)
- Especialización de clase (Persona → Autor)

Un lenguaje orientado a objetos debe favorecer la utilización de estas relaciones, ya que es una característica de este tipo de lenguajes, aunque no son exclusivos de ellos.

La abstracción es común a prácticamente todos los paradigmas de programación. El encapsulado aparece en los lenguajes modulares.

El polimorfismo en los lenguajes funcionales.

- Abstracción → Consiste en considerar sólo aquellos aspectos de un problema que son importantes desde cierto punto de vista.

- Enfoque en el modelado orientado a objetos de un sistema → Centrarse en qué es y qué hace un objeto antes de decidir cómo debe implementarse.

La abstracción aumenta nuestra libertad a la hora de tomar decisiones, evitando compromisos prematuros con los detalles concretos del sistema que se está modelando.

- Niveles de abstracción → Ayudan a estructurar la complejidad intrínseca que poseen los sistemas del mundo real.

- Proceso de abstracción → Donde se decide qué características y comportamiento se deben retener en el modelo.

Un modelo que retiene más de lo necesario puede dar lugar a redundancias y ambigüedades. Un modelo que no retiene lo suficiente puede carecer de utilidad o incumplir los requisitos exigidos. La abstracción es el principio fundamental que se encuentra tras la reutilización.

Sólo se puede reutilizar un componente si en él se ha abstraído la esencia de un conjunto de elementos del mundo real que aparecen con ligeras variantes.

- Encapsulamiento → Reunir, dotando de una cierta estructura, a todos los elementos que a un cierto nivel de abstracción se pueden considerar pertenecientes a una misma entidad.

- Proceso de encapsulamiento → Agrupar datos y operaciones relacionadas bajo una misma unidad de programación (clase). Esto permite aumentar la cohesión de los componentes del sistema.

No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.
Reservados todos los derechos.

WUOLAH
Participa en competiciones de programación. Encuentra el trabajo de tus sueños con NUWE

Cheetos
MÁNCHATE LOS DEDOS CON CHEETOS
Y ESCAQUÉATE
DE TOMAR APUNTES HOY

- Clases desde el punto de vista del encapsulamiento → Unidad de programación que encapsula datos y operaciones.

- Ocultación de datos → Permite separar el aspecto de un componente, que viene determinado por su interfaz con el exterior, de sus detalles internos de implementación.

- Interfaz del componente → Representa un contrato de prestación de servicios entre él y los demás componentes del sistema.

Los clientes de un componente sólo necesitan conocer los servicios que este ofrece, no como estos están realizados internamente.
Esto permite disminuir las interdependencias entre los componentes relacionados del sistema →
→ Reduce el acoplamiento entre ellos.

Para no "empañar" la visión externa de una estructura de datos o de funciones auxiliares que los clientes tienen de ellos, debemos ocultar los detalles irrelevantes para su uso y dotarlo de un mecanismo que permita decidir qué es lo que puede o no verse desde el exterior.

En general la ocultación de datos implica que no se deba permitir que los datos de un objeto sean modificados directamente desde el exterior, sino mediante la especificación de ocultación en la definición de la clase y proporcionando operaciones adicionales que permitan observar y modificar los datos de interés de una manera controlada (interfaz).

- Generalización → Consiste en no resolver nunca un problema concreto sin pensar antes en que probablemente sea un caso particular de un problema mucho más general.

La resolución de un problema general implica la resolución de muchos problemas derivados (además de poder haber podido resolver la particularización).

La generalización impulsa a compartir información, disminuyendo la redundancia.

- Herencia de especialización (proceso inverso generalización) → Permite definir nuevas clases a partir de otras ya existentes, de manera que presenten las características y comportamientos de éstas más (generalmente) otras adicionales.

- Polimorfismo → Capacidad de un componente para interpretar una solicitud de servicio de maneras distintas según el contexto. Si dicho componente se adapta a dicha especificación se denomina componente polimórfico.

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.
WUOLAH

CUESTIONARIO 1: BATERÍA DE PREGUNTAS

1) ¿Qué eficiencia se obtiene con la POO en comparación con la estructurada?

a) Siempre resulta, al menos, un orden de magnitud menos eficiente.
b) Nunca se llega a conseguir la misma eficiencia.
c) Puede llegar a ser igual de eficiente si se compila a un lenguaje de una máquina virtual.
d) Puede llegar a ser igual de eficiente.

2) ¿Limita el principio de ocultación de la información la comunicación entre los objetos?

a) No supone limitación , siempre se puede acceder directamente a los datos.
b) Sí, para objetos de una misma clase.
c) Sí, para objetos de clases distintas.
d) No supone limitación, en tanto en cuanto se comunican a través de su interfaz pública.

3) Las excepciones …

a) Se pueden sustituir por asertos si el lenguaje no posee excepciones.
b) Evitan la aparición de errores durante la ejecución .
c) Permiten gestionar situaciones excepcionales que pueden surgir durante la ejecución.
d) Son un mecanismo exclusivo de la POO.

4) ¿Cuál es la diferencia entre clase y objeto?

a) Las clases se pueden agrupar para formar otras clases, a diferencia de los objetos.
b) Las clases poseen métodos y atributos, a diferencia de los objetos.
c) Las clases poseen operaciones y datos, a diferencia de los objetos.
d) Una clase agrupa un conjunto de objetos, por lo que un objeto es una instancia (o ejemplar) de una clase.

5) El encapsulamiento y la ocultación de información …

a) Se definen al nivel de abstracción de una clase.
b) Son principios de la POO, pero se trata de principios ortogonales.
c) No poseen relación alguna.
d) Se definen al nivel de abstracción de un objeto aislado.

6) Diferencia entre compatibilidad y portabilidad del software:

a) La compatibilidad implica al código y a sus datos, mientras que la portabilidad solo implica al código.
b) La compatibilidad facilita la combinación de distintos componentes, mientras que la portabilidad permite ejecutar un software en entornos distintos.
c) La portabilidad está garantizada si las interfaces de los distintos componentes son compatibles, pero no a la inversa.
d) La compatibilidad solo implica al código , no a sus datos, mientras que la portabilidad implica considerar ambos aspectos.

7) ¿Cuál es el elemento central en el desarrollo de software mediante una metodología orientada a objetos?

a) Los asertos.
b) La herencia y el polimorfismo.
c) Las clases y sus relaciones.
d) Los objetos individuales.

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Participa en competiciones de programación. Encuentra el trabajo de tus sueños con NUWE

8) ¿En qué medida afecta la eficiencia a la corrección de un programa?
a) Un programa será más correcto cuanto más eficientemente use los recursos computacionales.
b) La eficiencia es un factor primordial para la corrección de un programa.
c) La corrección de un programa no depende de su eficiencia.
d) En gran medida, tanto que un programa ineficiente no puede ser correcto.

9) Factores de calidad que contribuyen a la fiabilidad de los programas.
a) Únicamente la corrección, son términos sinónimos.
b) La facilidad de manejo a través de una interfaz de usuario bien diseñada, así como la facilidad de modificación para aumentar su funcionalidad.
c) La extensibilidad y la protección de la integridad contra accesos o modificaciones sin autorización.
d) La fiabilidad de un programa se caracteriza por su corrección y robustez.

10) ¿Cuál es la diferencia fundamental entre el paradigma de la POO y el de la programación estructurada?
a) La POO prescinde del concepto de módulo, pues dispone de objetos.
b) La POO prescinde de la mayoría de las construcciones de la programación estructurada.
c) La POO une encapsulamiento y ocultación de información en un módulo de programación.
d) La POO posee excepciones.

11) Los asertos…
a) Son un mecanismo exclusivo de la POO.
b) Evitan la aparición de errores durante la ejecución.
c) No pueden emplearse como invariantes de clase.
d) Permiten comprobar el incumplimiento de una especificación durante la ejecución.

12) La principal ventaja que proporciona la herencia a las tareas de programación es…
a) Permitir la generalización mediante la definición de clases genéricas parametrizadas (plantillas).
b) Potenciar la reutilización de componentes mediante especialización.
c) Permitir la generalización de clases para reducir el acoplamiento.
d) Favorecer la abstracción para disminuir el código redundante

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Participa en competiciones de programación. Encuentra el trabajo de tus sueños con NUWE